



SIGNALS PROJECT

M1 & M2



Nour Eldin Mohamed	58-0359
Abdullah Essam saber	58-0340
Abdelrahman Mohamed	58-4203

Brief:-

1. we generated a song using the formula given in the project description while alternating from 3rd octave & 4th octave frequencies as if a piano is played
2. Random noise is added to the background where ,two random frequencies are selected within a certain range to generate the noise.
3. The Fourier Transform (FFT) is applied to both the original signal and the signal with noise
4. -we did noise cancelation where ,the frequencies of the noise peaks are identified, and waves with these frequencies are subtracted from the noisy signal to cancel out the noise.
5. we played both the original and filtered signals using the sounddevice library.
6. -we plotted the song, the sing with noise and the filtered song in the time domain &the frequency domain using matplotlib library.

Code:-

```

4
5 @author: a
6 """
7
8 import numpy as np
9 import matplotlib.pyplot as plt
10 from scipy.fftpack import fft
11 import sounddevice as sd
12
13
14
15
16 t = np.linspace(0,3,12*1024)
17
18 x = (np.sin(2*np.pi*440*t)*(np.heaviside(t,1)-np.heaviside(t-0.4,1)))
19 x = x + (np.sin(2*np.pi*220*t)*(np.heaviside(t-0.5,1)-np.heaviside(t-0.5-0.3,1)))
20 x = x + (np.sin(2*np.pi*196*t)*(np.heaviside(t-1,1)-np.heaviside(t-1-0.25,1)))
21 x = x + (np.sin(2*np.pi*392*t)*(np.heaviside(t-1.5,1)-np.heaviside(t-1.5-0.1,1)))
22 x = x + (np.sin(2*np.pi*440*t)*(np.heaviside(t-2.5,1)-np.heaviside(t-2.5-0.75,1)))
23
24 fn1, fn2 = np.random.randint(0, 512, 2)
25
26 noise = np.sin(2*np.pi*fn1*t) + np.sin(2*np.pi*fn2*t)
27
28 xn = x + noise
29
30 N = 3*1024
31
32 f = np.linspace(0, 512, int(N/2))
33
34 xf = (2/N) * np.abs(fft(x)[0:int(N/2)])
35
36 xnf = (2/N) * np.abs(fft(xn)[0:int(N/2)])
37
38 peak = np.max(xf)
39
40 tmp = []
41
42 for i in range(0, len(xnf)):
43     if xnf[i] > peak:
44         tmp += [i]
45
46 fn1_dash, fn2_dash = round(f[tmp[0]]), round(f[tmp[1]])
47
48 x_filtered = xn - (np.sin(2 * np.pi * fn1_dash * t) + np.sin(2 * np.pi * fn2_dash * t))
49
50 xnff = (2/N) * np.abs(fft(x_filtered)[0:int(N/2)])
51

```

```

52
53
54 sd.play(x, 3*1024)
55 sd.play(x_filtered, 4*1024)
56
57 plt.subplot(3, 2, 1)
58 plt.plot(t, x)
59 plt.title("Song")
60 plt.subplot(3, 2, 2)
61 plt.plot(t, xn)
62 plt.title("Song_with_Noise")
63 plt.subplot(3, 2, 3)
64 plt.plot(f, xf)
65 plt.title("Song_Frequency")
66 plt.subplot(3, 2, 4)
67 plt.plot(f, xnf)
68 plt.title("Song_with_Noise_Frequency")
69 plt.subplot(3, 2, 5)
70 plt.plot(t, x_filtered)
71 plt.title("Filtered_Song")
72 plt.subplot(3, 2, 6)
73 plt.plot(f, xnff)
74 plt.title("Filtered_Song_Frequency")
75

```

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.fftpack import fft
import sounddevice as sd
```

```
t = np.linspace(0,3,12*1024)
```

```
x = (np.sin(2*np.pi*440*t)*(np.heaviside(t,1)-np.heaviside(t-0.4,1)))
x = x + (np.sin(2*np.pi*220*t)*(np.heaviside(t-0.5,1)-np.heaviside(t-0.5-0.3,1)))
x = x + (np.sin(2*np.pi*196*t)*(np.heaviside(t-1,1)-np.heaviside(t-1-0.25,1)))
x = x + (np.sin(2*np.pi*392*t)*(np.heaviside(t-1.5,1)-np.heaviside(t-1.5-0.1,1)))
x = x + (np.sin(2*np.pi*440*t)*(np.heaviside(t-2.5,1)-np.heaviside(t-2.5-0.75,1)))
```

```
fn1, fn2 = np.random.randint(0, 512, 2)
```

```
noise = np.sin(2*np.pi*fn1*t) + np.sin(2*np.pi*fn2*t)
```

```
xn = x + noise
```

```
N = 3*1024
```

```
f = np.linspace(0, 512, int(N/2))
```

```
xf = (2/N) * np.abs(fft(x)[0:int(N/2)])
```

```
xnf = (2/N) * np.abs(fft(xn)[0:int(N/2)])
```

```
peak = np.max(xf)
```

```
tmp = []
```

```
for i in range(0, len(xnf)):
```

```
    if xnf[i] > peak:
```

```
        tmp += [i]
```

```
fn1_dash, fn2_dash = round(f[tmp[0]]), round(f[tmp[1]])
```

```
x_filtered = xn - (np.sin(2 * np.pi * fn1_dash * t) + np.sin(2 * np.pi * fn2_dash * t))
```

```
xnff = (2/N) * np.abs(fft(x_filtered)[0:int(N/2)])
```

```
sd.play(x, 3*1024)
```

```
sd.play(x_filtered, 4*1024)
```

```
plt.subplot(3, 2, 1)
```

```
plt.plot(t, x)
```

```
plt.title("Song")
```

```
plt.subplot(3, 2, 2)
```

```
plt.plot(t, xn)
```

```
plt.title("Song_with_Noise")
```

```
plt.subplot(3, 2, 3)
```

```
plt.plot(f, xf)
plt.title("Song_Frequency")
plt.subplot(3, 2, 4)
plt.plot(f, xnf)
plt.title("Song_with_Noise_Frequency")
plt.subplot(3, 2, 5)
plt.plot(t, x_filtered)
plt.title("Filtered Song")
plt.subplot(3, 2, 6)
plt.plot(f, xnff)
plt.title("Filtered_Song_Frequency")
```

6 Figures:-

